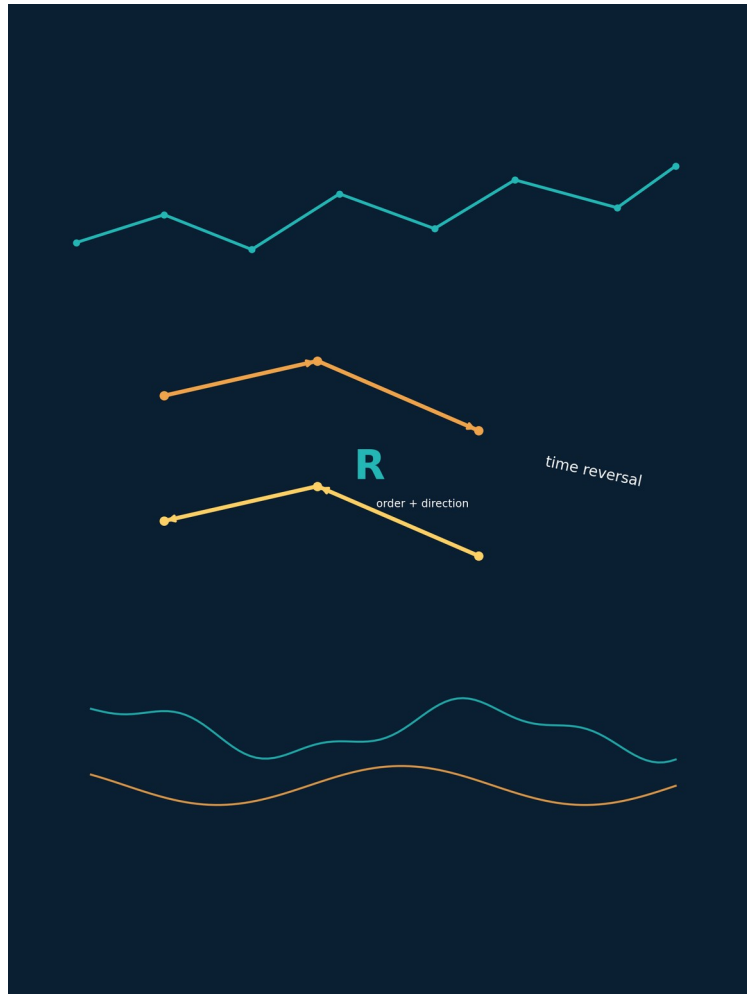




PHASE 6 STANDALONE HANDBOOK

Ordered Two-Step Sequences and Time-Reversal Analysis

Time Irreversibility in Cell Motility

A cell-by-cell scientific, mathematical, biophysical, Python, and decision-making record of Phase 6

VALIDATED PHASE SNAPSHOT

Executed notebook: `notebooks/06_phase6_sequence_irreversibility_MSC01.ipynb` • Exact sequences: 392 from 32 cells • Paired feature analysis: 365 sequences from 31 cells • Final null result: neither raw nor co-moving observed asymmetry exceeded its null 95th percentile • Automated tests: 9 passed.

Prepared as a standalone continuation companion to the Phase 0–4 Master Handbook and the Phase 5 Standalone Handbook

Document purpose, scope, and evidence standard

This standalone handbook reconstructs Phase 6 in enough detail that a fresh ChatGPT session, collaborator, or future version of the user can understand exactly what was done, why it was done, what every notebook cell contributed, what outputs were observed, and how each result changed the next decision.

DOCUMENTATION BOUNDARY

Phase 6 is an exploratory scientific training phase and a GitHub portfolio artifact. The analysis is scientifically disciplined and reproducible, but it is not framed as a publication-grade proof of irreversibility, a thermodynamic entropy-production measurement, or a requirement to obtain a positive result.

Evidence item	What it establishes
Executed Notebook 06	Authoritative cell order, code logic, and saved notebook outputs.
Recorded screenshots in this chat	Exact numerical tables and validation messages used during interpretation.
Phase 5 standalone handbook	Immediate prior scientific context: displacement inversion, KDE, bootstrap/null controls, and project mindset.
Automated test run	<code>`python -m pytest -q`</code> returned 9 passed in 0.55 s before the Phase 6 commit workflow.
Git history recorded in-session	Phase 6 notebook commit <code>`f26276a`</code> ; Phase 5 handbook commit <code>`83f32f0`</code> ; branch <code>`phase-6-sequence-irreversibility`</code> was pushed before PR/merge workflow.

EXACT STOPPING POINT

Phase 6 is scientifically complete. The next phase should extend the representation or temporal depth only if that adds interpretability or detection power; Phase 6 itself should not be reopened merely to force a positive time-arrow result.

[↑ Back to Contents](#)

Contents

- [1. Phase 6 handoff from Phase 5](#)
- [2. Mathematical and biophysical foundations](#)
- [3. Data inputs, invariants, and phase design](#)
- [4. Cells 1–7: exact two-step sequence construction](#)
- [5. Cells 8–14: time-reversal operator and time-odd observables](#)
- [6. Cells 15–20: paired feature-space construction](#)
- [7. Cells 21–28: scaled KDE and baseline sequence asymmetry](#)
- [8. Cells 29–35: robustness, bootstrap, and null control](#)
- [9. Cell 36: final Phase 6 scientific conclusion](#)
- [10. Integrated Phase 6 results and interpretation](#)
- [11. Decision logic and project mindset](#)
- [12. Python and scientific-computing concepts learned](#)
- [13. Reproducibility, testing, and Git workflow](#)
- [14. Phase 7 handoff and next decision space](#)
- [**Appendix A. Key Phase 6 code listings**](#)
- [Appendix B. Exact numerical result ledger](#)
- [Appendix C. Glossary and notation](#)
- [Appendix D. New-chat handoff prompt](#)

Every major section deliberately starts on a new page. Unused space at the end of a section is intentional so that section boundaries remain visually obvious.

1. Phase 6 handoff from Phase 5

Phase 5 asked whether the distribution of a single displacement vector was invariant under spatial inversion. That was a useful directional-asymmetry diagnostic, but a single displacement does not contain temporal ordering. Phase 6 therefore changed the scientific object: the fundamental observation became an ordered pair of consecutive displacement vectors.

Phase 5: $\Delta r \leftrightarrow -\Delta r$ (single-displacement spatial inversion)

Phase 6: $S = (\Delta r_1, \Delta r_2) \leftrightarrow R(S) = (-\Delta r_2, -\Delta r_1)$ (ordered time reversal)

Inherited validated fact	Phase 6 consequence
Clean raw and co-moving tracks share the same cell/frame/time structure.	Sequence identities can be paired exactly across representations.
Frame interval = 20 min.	A two-step sequence spans 40 min and requires three consecutive observations.
Exact-lag code in `src/steps.py` is authoritative.	Phase 6 reuses exact one-frame steps rather than reintroducing row-shift logic.
Phase 5 showed positive single-step inversion metrics, but finite-sample nulls mattered.	Phase 6 must include an explicit time-reversal-symmetrized null before interpreting any positive KL value.
Project is educational, exploratory, reproducible, and portfolio-oriented.	Stop when the scientific question is answered; do not chase anomalies or significance for its own sake.

KEY CONCEPTUAL UPGRADE

Correct time reversal is not $(-\Delta r_1, -\Delta r_2)$. It must reverse both vector direction and temporal order: $(-\Delta r_2, -\Delta r_1)$. This distinction is the central conceptual reason Phase 6 is more temporally meaningful than Phase 5.

[↑ Back to Contents](#)

2. Mathematical and biophysical foundations

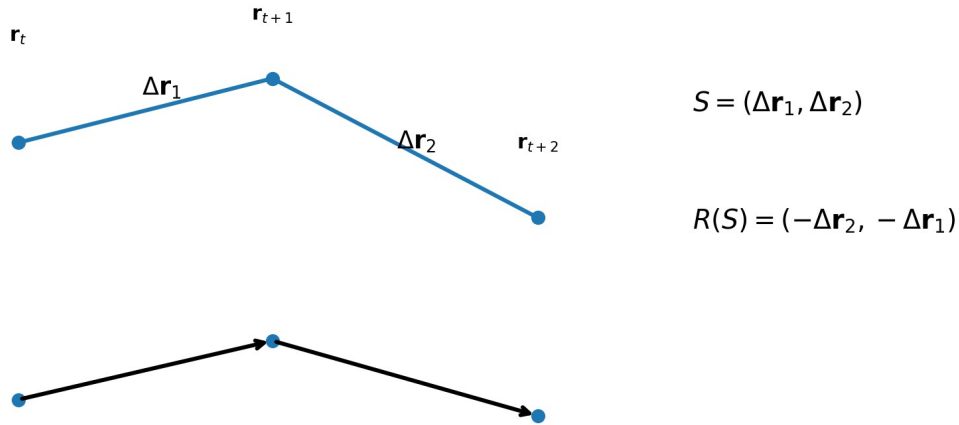
2.1 Ordered two-step sequence

For positions $r(t)$, $r(t+1)$, and $r(t+2)$, define two consecutive one-frame displacements. The sequence contains the order information that a single displacement lacks.

$$\Delta r_1 = r(t+1) - r(t)$$

$$\Delta r_2 = r(t+2) - r(t+1)$$

$$S = (\Delta r_1, \Delta r_2)$$



Correct reversal: reverse step order AND vector direction

Figure 1. Forward two-step sequence and the correct time-reversal operation.

2.2 Time reversal and involution

The time-reversal map must satisfy an involution property: applying it twice returns the original sequence. This became a computational unit-style validation inside the notebook rather than a purely verbal assumption.

$$R(S) = (-\Delta r_2, -\Delta r_1)$$

$$R(R(S)) = S$$

2.3 Interpretable time-odd features

The full sequence lives in four dimensions: $(\Delta x_1, \Delta y_1, \Delta x_2, \Delta y_2)$. With only a few hundred sequences, direct four-dimensional density estimation would be data-hungry and difficult to interpret. Phase 6 therefore used a two-dimensional projection chosen because both components change sign under time reversal.

$$m_1 = \|\Delta \mathbf{r}_1\|, \quad m_2 = \|\Delta \mathbf{r}_2\|, \quad \Delta m = m_2 - m_1$$

$$\theta = \text{atan2}(\Delta x_1 \Delta y_2 - \Delta y_1 \Delta x_2, \Delta x_1 \Delta x_2 + \Delta y_1 \Delta y_2)$$

$$z = (\Delta m, \sin \theta), \quad R(z) = -z$$

The turning angle is undefined when either step has zero magnitude. Those cases were not investigated as tracking anomalies; they were treated as mathematically invalid turning-angle observations and excluded from the paired feature analysis.

2.4 Why sin(theta) rather than theta in ordinary KDE?

Angles are circular: $+\pi$ and $-\pi$ are adjacent directions even though their numeric values are far apart. Ordinary Euclidean KDE would mishandle that boundary. Using $\sin(\theta)$ retains the signed turning component needed for this exploratory time-odd

representation and obeys $\sin(-\theta) = -\sin(\theta)$. It does not encode every property of the original angle, so the representation remains intentionally reduced.

2.5 Scaling without centering

Δm has units of micrometres, whereas $\sin(\theta)$ is dimensionless and bounded. The two components therefore require scale normalization before isotropic 2D KDE. The features were divided by robust pooled scales but were not mean-centered. Centering would shift the physical time-reversal origin away from zero and would break the simple transformation $z \rightarrow -z$.

2.6 KDE and KL asymmetry

$$I_{\text{seq}} = \int p(z) \log[p(z) / p(-z)] dz$$

This KL-based quantity is non-negative in the ideal continuous setting, but a positive estimate from a finite sample is not automatically evidence of a time arrow. KDE smoothing, sample size, and irregular sampling can generate positive background asymmetry. Phase 6 therefore treats the time-reversal-symmetrized null as the decisive interpretive reference.

2.7 Biophysical interpretation boundary

WHAT PHASE 6 CAN SAY

It can test whether a specific, interpretable two-step feature representation carries detectable forward-versus-reversed asymmetry beyond the chosen finite-sample/KDE null background.

WHAT PHASE 6 CANNOT SAY

It cannot prove that the underlying cellular dynamics are time-reversible; it cannot measure thermodynamic entropy production; it cannot exhaust all possible sequence-level irreversibility because the 4D sequence was projected into two features.

[↑ Back to Contents](#)

3. Data inputs, invariants, and phase design

Item	Phase 6 value / decision
Raw input	`data/MS01_tracks_clean.csv`
Co-moving input	`data/MS01_tracks_comoving.csv`
Track columns	cell_id, frame, t_min, x_um, y_um
Input shape	474 rows × 5 selected columns in each representation
Frame interval	20 min
Sequence length	2 one-frame steps = 3 consecutive positions = 40 min span
Sequence construction	Join exact one-frame steps on the same cell and exact middle frame
Raw/co-moving pairing	Same temporal sequence structure required
Primary analysis subset	Paired valid-turn sequences only
Random seed	2026
Bootstrap replicates	100 cell-level resamples
Null simulations	100 sign-randomization runs per representation = 200 stored rows
KDE grid	Symmetric around the time-reversal origin
Interpretation	Exploratory, null-calibrated, no entropy-production claim

Why 392 sequences instead of the earlier 398 tau=2 displacements?

An exact tau=2 displacement only requires an observation at t and t+2. A two-step ordered sequence additionally requires the middle observation at t+1. Phase 6 therefore found 392 valid consecutive sequences, six fewer than the 398 exact tau=2 endpoint displacements from earlier phases. This difference is expected and was not treated as an error.

DESIGN DISCIPLINE

Phase 6 intentionally stayed compact. The notebook stopped at 36 cells because the key construction, mathematical validations, baseline estimator, bandwidth sensitivity, cell-level bootstrap, null control, and conservative conclusion were all present. No additional diagnostics were opened once the null result answered the main question.

[↑ Back to Contents](#)

4. Cells 1–7: exact two-step sequence construction

This block creates the new scientific object for Phase 6. It deliberately reuses validated exact one-frame displacement logic and requires exact temporal continuity through the middle frame.

Cell 1 — Phase title and goal

Type / role	Markdown / scope definition
Code or content summary	Defined the scientific transition from single displacements to ordered two-step sequences and stated the correct reversed sequence.
Stored output	No executable output.
Interpretation / decision	Locked the Phase 6 question: distinguish observed forward two-step sequences from their correctly time-reversed counterparts.
Concepts taught / reinforced	Scientific scoping; mathematical notation; explicit interpretation boundary.

Cell 2 — Imports and constants

Type / role	Code / environment setup
Code or content summary	Ran `_path.py`; imported NumPy, pandas, Matplotlib; imported validated track helpers; set 20 min/frame, sequence length 2, random seed 2026.
Stored output	Printed `Phase 6 environment loaded.` and the local project root.
Interpretation / decision	Reused validated project infrastructure instead of rebuilding basic trajectory logic.
Concepts taught / reinforced	Imports; constants; reproducibility seed; modular code reuse.

Cell 3 — Load raw and co-moving trajectories

Type / role	Code / input loading and validation
Code or content summary	Loaded the two CSV inputs, selected the five authoritative columns, sorted by cell/frame, reset indices, and called `validate_tracks`.
Stored output	Raw tracks shape: (474, 5). Co-moving tracks shape: (474, 5).
Interpretation / decision	Confirmed both representations contain the same number of trajectory observations and are ready for paired temporal construction.
Concepts taught / reinforced	Pathlib-style paths; pandas selection/sort/reset; validation.

Cell 4 — Exact temporal continuity

Type / role	Markdown / scientific invariant
Code or content summary	Explained why t and $t+2$ alone are insufficient for an ordered two-step sequence; frame $t+1$ must exist.
Stored output	No executable output.
Interpretation / decision	Set the rule that sequence construction must explicitly join $t \rightarrow t+1$ and $t+1 \rightarrow t+2$.
Concepts taught / reinforced	Difference between endpoint lag and contiguous sequence; gap awareness.

Cell 5 – Build exact two-step sequences

Type / role	Code / core constructor definition
Code or content summary	Defined `build_two_step_sequences`: compute exact tau=1 steps, rename first/second step columns, merge on cell_id and frame_mid, validate one-to-one, check both one-frame transitions.
Stored output	No printed output; function definition stored.
Interpretation / decision	Adopted merge-based exact continuity as the authoritative construction method.
Concepts taught / reinforced	Functions; DataFrame rename/select/copy; merge; boolean validation; RuntimeError.

Cell 6 – Construct raw and co-moving sequences

Type / role	Code / execution and first results
Code or content summary	Applied the constructor to both representations, built `sequence_summary`, displayed summary and first raw rows.
Stored output	392 sequences, 32 cells in each representation; min start frame 0; max end frame 47; first rows showed exact 10→11→12, 11→12→13, etc.
Interpretation / decision	Established the primary sequence count and confirmed raw/co-moving temporal structures can be compared.
Concepts taught / reinforced	List of dictionaries; DataFrame construction; nunique; head.

Cell 7 – Structural validation

Type / role	Code / paired invariant tests
Code or content summary	Asserted raw and co-moving cell/frame/time columns are identical; asserted 2-frame span and 40-min physical span.
Stored output	`Phase 6 sequence-construction validation: PASSED`.
Interpretation / decision	Sequence construction checkpoint passed; analysis could safely move to time reversal.
Concepts taught / reinforced	`pd.testing.assert_frame_equal`; `assert`; `np.allclose`.

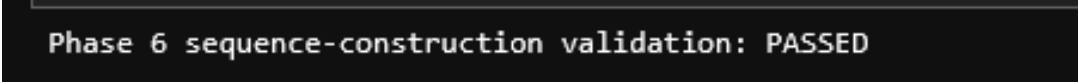
```
display(sequence_summary)
```

```
display(
    raw_sequences.head()
)
```

	representation	n_sequences	n_cells	min_start_frame	max_end_frame
0	raw	392	32	0	47
1	comoving	392	32	0	47

	cell_id	frame_start	t_start_min	frame_mid	t_mid_min	dx1_um	dy1_um	frame_end	t_end_min	dx2_um	dy2_um	sequence_span.1
0	0	10	200.0	11	220.0	-5.091252	3.172796	12	240.0	-3.720196	2.155006	
1	0	11	220.0	12	240.0	-3.720196	2.155006	13	260.0	1.393048	0.534448	
2	0	12	240.0	13	260.0	1.393048	0.534448	14	280.0	0.919362	0.004737	
3	1	0	0.0	1	20.0	-1.043849	0.011084	2	40.0	-0.439572	-0.169187	
4	1	1	20.0	2	40.0	-0.439572	-0.169187	3	60.0	-1.189467	0.122024	

Figure 2. Cell 6 stored output: 392 sequences from 32 cells in both representations and the first five raw sequences.



Phase 6 sequence-construction validation: PASSED

Figure 3. Cell 7 sequence-construction validation checkpoint.

[↑ Back to Contents](#)

5. Cells 8–14: time-reversal operator and time-odd observables

This block moves from temporal construction to mathematical correctness. The reversal convention is tested directly, and the initial time-odd observables are introduced before any density estimator is fitted.

Cell 8 – The time-reversal operator

Type / role	Markdown / mathematical definition
Code or content summary	Defined $S=(\Delta r_1, \Delta r_2)$, $R(S)=(-\Delta r_2, -\Delta r_1)$, and involution $R(R(S))=S$.
Stored output	No executable output.
Interpretation / decision	Locked the physically correct reversal convention before statistical analysis.
Concepts taught / reinforced	Involution; ordered transformation; time-reversal semantics.

Cell 9 – Convert sequences to NumPy arrays

Type / role	Code / representation helper
Code or content summary	Defined <code>`get_sequence_array`</code> returning columns <code>[dx1, dy1, dx2, dy2]</code> as an <code>(n,4)</code> floating array.
Stored output	No printed output.
Interpretation / decision	Created a compact numerical representation suitable for vectorized operations.
Concepts taught / reinforced	NumPy arrays; column order as an invariant; dtype control.

Cell 10 – Define time-reversal function

Type / role	Code / operator implementation
Code or content summary	Defined <code>`reverse_two_step_sequences`</code> ; checked input shape; returned <code>[-dx2, -dy2, -dx1, -dy1]</code> with <code>`np.column_stack`</code> .
Stored output	No printed output.
Interpretation / decision	Translated the mathematical operator directly into testable code.
Concepts taught / reinforced	Array slicing; shape validation; <code>`np.column_stack`</code> ; <code>ValueError</code> .

Cell 11 – Validate reversal and involution

Type / role	Code / mathematical-computational test
Code or content summary	Constructed forward, reversed, and double-reversed raw arrays; checked shape and equality.
Stored output	Shape (392,4); involution PASSED; first forward sequence <code>[-5.09125217, 3.17279647, -3.72019571, 2.15500559]</code> ; reverse <code>[3.72019571, -2.15500559, 5.09125217, -3.17279647]</code> ; double reverse returned original.
Interpretation / decision	The implemented R operator was accepted as correct.
Concepts taught / reinforced	Vectorized transforms; numerical equality; property-based reasoning.

Cell 12 – Interpretable time-odd features

Type / role	Markdown / dimensionality decision
Code or content summary	Introduced Δm and signed turning angle as a low-dimensional projection of the 4D sequence; showed both flip sign under time reversal.
Stored output	No executable output.
Interpretation / decision	Chose interpretability and sample-size realism over direct 4D KDE.
Concepts taught / reinforced	Curse of dimensionality; feature engineering; time-odd observables.

Cell 13 – Compute two-step features

Type / role	Code / feature extractor
Code or content summary	Defined magnitudes via <code>`np.hypot`</code> , $\Delta m = m_2 - m_1$, cross/dot terms, signed angle via <code>`np.arctan2`</code> ; invalidated turning angle when either step length is zero.
Stored output	No printed output.
Interpretation / decision	Zero-length-step angles are mathematically undefined and should be marked missing, not forensically investigated.
Concepts taught / reinforced	Vector norms; dot/cross terms; <code>atan2</code> ; boolean masks; NaN handling.

Cell 14 – Test feature reversal

Type / role	Code / invariant validation
Code or content summary	Computed features for forward and reversed raw arrays; asserted Δm and θ reverse sign for valid turns.
Stored output	<code>`Feature time-reversal tests: PASSED`</code> ; 376 / 392 raw sequences had valid turning angles.
Interpretation / decision	Confirmed the chosen observables behave as required under the validated R operator.
Concepts taught / reinforced	Boolean pair masks; signed-feature validation.

```
Raw sequence-array shape: (392, 4)
Time-reversal involution test: PASSED

First forward sequence:
[-5.09125217  3.17279647 -3.72019571  2.15500559]

Its time reverse:
[ 3.72019571 -2.15500559  5.09125217 -3.17279647]

Double reverse:
[-5.09125217  3.17279647 -3.72019571  2.15500559]
```

Figure 4. Cell 11: exact numerical example showing forward, reversed, and double-reversed sequence.

```
Feature time-reversal tests: PASSED
Valid turning-angle sequences: 376 / 392
```

Figure 5. Cell 14: feature sign-reversal tests passed; 376/392 raw sequences had defined turning angles.

[↑ Back to Contents](#)

6. Cells 15–20: paired feature-space construction

The second design choice of Phase 6 is made here: replace the circular angle by $\sin(\theta)$, then use only sequences whose turning information is valid in both raw and co-moving representations.

Cell 15 — Analysis feature space

Type / role	Markdown / circular-variable decision
Code or content summary	Explained why ordinary Euclidean KDE should not use raw θ because $\pm\pi$ are adjacent; selected $\sin(\theta)$, preserving sign reversal.
Stored output	No executable output.
Interpretation / decision	Primary feature vector became $z=(\Delta m, \sin\theta)$; retained θ only for interpretation.
Concepts taught / reinforced	Circular variables; representation choice; $\sin(-\theta)=-\sin\theta$.

Cell 16 — Build raw/co-moving feature tables

Type / role	Code / metadata-preserving feature construction
Code or content summary	Computed feature tables for both representations, added signed-turn sine, representation labels, cell_id and frame_start metadata.
Stored output	No printed output.
Interpretation / decision	Prepared paired features while retaining cell identity for later cluster bootstrap.
Concepts taught / reinforced	Column creation; metadata alignment; NumPy-to-pandas integration.

Cell 17 — Validate feature availability

Type / role	Code / raw vs co-moving validity audit
Code or content summary	Checked table lengths and paired identifiers; printed valid/invalid turning counts.
Stored output	Raw valid 376; co-moving valid 380; raw invalid 16; co-moving invalid 12.
Interpretation / decision	Recognized that common-motion subtraction can change zero-length status; raw and co-moving valid-turn sets are not identical.
Concepts taught / reinforced	Boolean inversion $\sim$; paired metadata invariants.

Cell 18 — Create paired valid subset

Type / role	Code / fairness decision
Code or content summary	Defined intersection of valid-turn masks and created paired raw/co-moving analysis tables.
Stored output	365 / 392 paired valid sequences; 31 cells represented.
Interpretation / decision	Fixed the primary analysis sample at identical sequences in both representations rather than comparing different subsets.
Concepts taught / reinforced	Boolean AND masks; <code>.loc</code> ; <code>reset_index</code> ; paired design.

Cell 19 – Validate sine feature reversal

Type / role	Code / final representation test
Code or content summary	Computed reversed features, applied sine, and asserted sign reversal on valid pairs.
Stored output	`Signed-turn sine reversal test: PASSED`.
Interpretation / decision	Confirmed the final KDE feature representation obeys $z \rightarrow -z$ under time reversal.
Concepts taught / reinforced	Derived-feature validation; trigonometric identity in code.

Cell 20 – Summarize time-odd features

Type / role	Code / descriptive statistics
Code or content summary	Built a two-row summary of sequence count, cell count, mean/median Δm , mean/median $\sin\theta$ for raw and co-moving.
Stored output	Raw mean Δm 0.036846 μm , median ~ 0 ; mean $\sin\theta$ -0.012240, median 0. Co-moving mean Δm 0.025959 μm , median 0.0675526; mean $\sin\theta$ -0.014016, median 0.007877.
Interpretation / decision	Marginal means were close to zero; this did not justify declaring time symmetry because joint asymmetry can remain even with near-zero marginals.
Concepts taught / reinforced	Descriptive statistics; cancellation; distinction between marginal and joint structure.

```
Raw valid turns: 376
Co-moving valid turns: 380
Raw invalid turns: 16
Co-moving invalid turns: 12
```

Figure 6. Cell 17 raw/co-moving valid-turn counts.

```
Paired valid sequences: 365 / 392
Cells represented: 31
```

Figure 7. Cell 18 paired subset: 365 sequences from 31 cells.

```
Signed-turn sine reversal test: PASSED
```

Figure 8. Cell 19 signed-turn-sine reversal validation.

representation	n_sequences	n_cells	mean_delta_magnitude_um	median_delta_magnitude_um	mean_signed_turn_sine	median_signed_turn_sine
raw	365	31	0.036846	-2.842171e-14	-0.012240	0.000000
comoving	365	31	0.025959	6.755259e-02	-0.014016	0.000787

Figure 9. Cell 20 descriptive feature summary. Means/medians are close to zero, motivating joint rather than marginal analysis.

[↑ Back to Contents](#)

7. Cells 21–28: scaled KDE and baseline sequence asymmetry

This block defines the actual Phase 6 estimator. Features are pooled-scaled without centering, then a shared 2D KDE grid and a KL comparison between $p(z)$ and $p(-z)$ are used.

Cell 21 — Joint sequence-level asymmetry

Type / role	Markdown / core statistical question
Code or content summary	Defined comparison $p(z)$ versus $p(-z)$, explicitly linking inversion to ordered sequence reversal rather than single-step spatial inversion.
Stored output	No executable output.
Interpretation / decision	Moved from descriptive means to joint distribution analysis.
Concepts taught / reinforced	Joint probability density; representation-level irreversibility.

Cell 22 — Robust pooled feature scales

Type / role	Code / unit normalization
Code or content summary	Defined robust scale as $\min(\text{standard deviation}, \text{IQR}/1.34)$ over positive finite candidates; pooled raw and co-moving features.
Stored output	Δm scale 1.649157496997992; signed-turn-sine scale 0.49157537528456907.
Interpretation / decision	Used common scales for fair comparison and avoided mean centering to preserve reversal origin.
Concepts taught / reinforced	Robust scale; IQR; list comprehension; pooled normalization.

Cell 23 — Build scaled feature arrays

Type / role	Code / dimensionless coordinate construction
Code or content summary	Divided Δm and $\sin\theta$ by pooled scales; stacked into 2D arrays for each representation.
Stored output	Raw shape (365,2); co-moving shape (365,2).
Interpretation / decision	Established equal-dimensional, equal-sample feature arrays for KDE.
Concepts taught / reinforced	Broadcast division; dimensionless variables; <code>column_stack</code> .

Cell 24 — Validate reversal after scaling

Type / role	Code / transformed invariant
Code or content summary	Confirmed negation is still the time-reversal operation in scaled coordinates and double negation returns original.
Stored output	`Scaled feature reversal validation: PASSED`.
Interpretation / decision	Scaling was accepted as compatible with the reversal symmetry.
Concepts taught / reinforced	Invariant preservation under linear scaling.

Cell 25 — Define KDE helpers

Type / role	Code / density machinery
Code or content summary	Imported `KernelDensity`; defined $n^{(-1/6)}$ rule-of-thumb bandwidth, symmetric grid, and 2D Gaussian KDE evaluator.
Stored output	No printed output.
Interpretation / decision	Reused the Phase 5 KDE concept in a new, explicitly temporal feature space.
Concepts taught / reinforced	KDE; dimensional bandwidth rule; meshgrid; score_samples; exponentiating log density.

Cell 26 — Define sequence-asymmetry index

Type / role	Code / metric definition
Code or content summary	Normalized KDE over the grid, generated reversed density by array flipping, regularized with epsilon, and integrated $p \log(p/q)$.
Stored output	No printed output.
Interpretation / decision	Established a KL-based sequence-feature asymmetry metric while preserving finite-sample caution.
Concepts taught / reinforced	Discrete numerical integration; KL divergence; epsilon regularization; symmetric grid indexing.

Cell 27 — Compute baseline sequence asymmetry

Type / role	Code / primary estimate
Code or content summary	Computed shared baseline bandwidth, shared symmetric grid, KDEs and KL asymmetry for raw and co-moving.
Stored output	Bandwidth 0.374069. Raw grid mass 0.999935, asymmetry 0.107103 nats. Co-moving grid mass 0.999890, asymmetry 0.189430 nats.
Interpretation / decision	Grid coverage passed. Co-moving > raw was observed, but no biological interpretation was accepted before robustness/null checks.
Concepts taught / reinforced	Baseline estimation; shared hyperparameters; sanity checking probability mass.

Cell 28 — Visualize raw feature KDE

Type / role	Code / diagnostic visualization
Code or content summary	Evaluated raw KDE and plotted filled contours with zero axes in scaled Δm vs scaled $\sin\theta$ space.
Stored output	Stored contour plot with density concentrated near the origin and elongated mainly along the turning-sine axis.
Interpretation / decision	Plot was treated as qualitative context only; the null-calibrated metric remained the decision tool.
Concepts taught / reinforced	Contour plots; visual symmetry around origin; plot-vs-metric distinction.

Pooled delta-magnitude scale: 1.649157496997992
Pooled signed-turn-sine scale: 0.49157537528456907

Figure 10. Cell 22 pooled robust scales used to make the feature coordinates dimensionless.

Raw feature-array shape: (365, 2)
Co-moving feature-array shape: (365, 2)

Figure 11. Cell 23 scaled raw and co-moving feature-array shapes.

Scaled feature reversal validation: PASSED

Figure 12. Cell 24 scaled-feature reversal validation.

representation	n_sequences	bandwidth	grid_mass	sequence_asymmetry_nats
raw	365	0.374069	0.999935	0.107103
comoving	365	0.374069	0.999890	0.189430

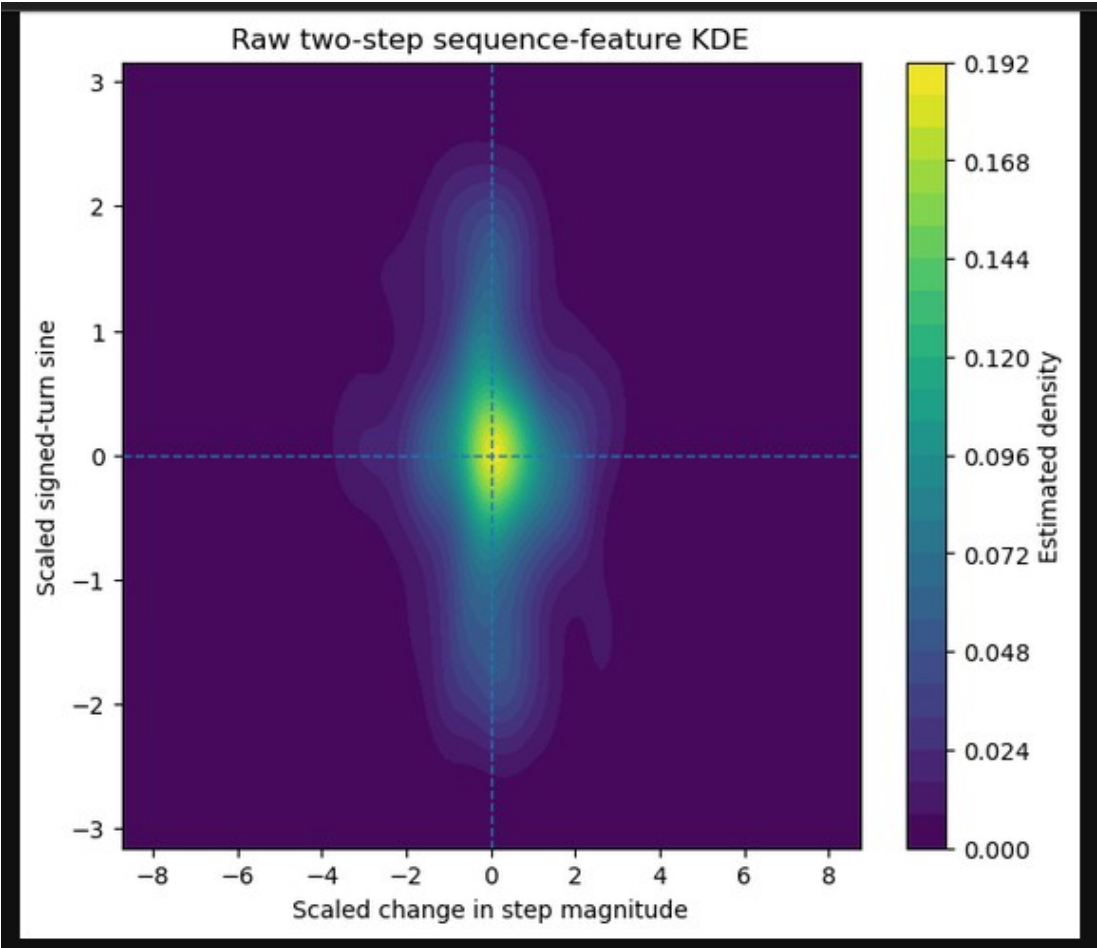


Figure 13. Cell 28 raw two-step sequence-feature KDE in scaled coordinates.

BASELINE DECISION

The co-moving point estimate (0.189 nats) exceeded the raw estimate (0.107 nats), but the project explicitly refused to interpret this as biological evidence before bandwidth, cell-bootstrap, and null calibration.

8. Cells 29–35: robustness, bootstrap, and null control

This block is deliberately short and decisive. It tests whether the baseline number is stable to smoothing, cell composition, and the finite-sample background expected after explicit time-reversal symmetrization.

Cell 29 — Final robustness plan

Type / role	Markdown / scope control
Code or content summary	Declared only three compact robustness checks: bandwidth sensitivity, cell-level bootstrap, time-reversal-symmetrized null.
Stored output	No executable output.
Interpretation / decision	Prevented Phase 6 from expanding into another large anomaly-audit notebook.
Concepts taught / reinforced	Analysis planning; stopping rule; exploratory scope.

Cell 30 — Bandwidth sensitivity

Type / role	Code / smoothing robustness
Code or content summary	Repeated metric at 0.5×, 1×, and 2× baseline bandwidth; pivoted raw/co-moving and calculated difference.
Stored output	0.5×: co-moving 0.658550, raw 0.484923; 1×: 0.189430 vs 0.107103; 2×: 0.036751 vs 0.019244.
Interpretation / decision	Ordering co-moving > raw persisted, but absolute metric changed strongly with smoothing; exact values are not intrinsic constants.
Concepts taught / reinforced	Sensitivity analysis; pivot; bandwidth dependence.

Cell 31 — Cell-level bootstrap

Type / role	Code / cluster uncertainty
Code or content summary	Resampled the 31 contributing cells with replacement; kept all sequences for each sampled cell; used fixed scales/grid/bandwidth; 100 replicates.
Stored output	Printed `Bootstrap replicates: 100`.
Interpretation / decision	Estimated cell-composition uncertainty without treating overlapping sequences as independent rows.
Concepts taught / reinforced	Cluster bootstrap; random generator; lists of blocks; `np.vstack`; paired resampling.

Cell 32 – Summarize bootstrap

Type / role	Code / interval estimation
Code or content summary	Computed bootstrap means and 2.5%/97.5% quantiles for raw, co-moving, and their difference.
Stored output	Raw mean 0.223156 [0.104131, 0.478895]; co-moving 0.298824 [0.166411, 0.559712]; difference 0.075668 [-0.033210, 0.184291].
Interpretation / decision	Because the difference interval crosses zero, `co-moving > raw` is not robust to cell composition.
Concepts taught / reinforced	Quantile intervals; paired-difference interpretation; no overclaim from sign of point estimate.

Cell 33 – Time-reversal-symmetrized null

Type / role	Code / finite-sample background
Code or content summary	For each null run, generated ± 1 signs and applied the same sign realization to paired raw and co-moving vectors; ran 100 simulations per representation.
Stored output	Printed `Null simulations: 200` (100 per representation).
Interpretation / decision	Constructed the decisive background model: systematic time directionality removed in expectation while empirical feature scale/structure retained.
Concepts taught / reinforced	Random sign flips; paired null design; symmetry in expectation.

Cell 34 – Compare observed with null

Type / role	Code / decisive inference table
Code or content summary	Grouped null runs to mean and q95, merged with observed values, computed observed-minus-null mean and Boolean `above_null_q95`.
Stored output	Raw observed 0.107103, null mean 0.190833, q95 0.264116, False. Co-moving observed 0.189430, null mean 0.185367, q95 0.261925, False.
Interpretation / decision	Neither representation exceeded the null 95th percentile. This became the central scientific conclusion of Phase 6.
Concepts taught / reinforced	groupby/agg; lambda quantile; merge with one-to-one validation; Boolean thresholding.

Cell 35 – Final computational validation

Type / role	Code / completion checkpoint
Code or content summary	Asserted array shapes, near-unity grid masses, replicate counts, and finite bootstrap values.
Stored output	`Phase 6 computational validation: PASSED`.
Interpretation / decision	All planned Phase 6 computations and integrity checks passed; no extra analyses were needed.
Concepts taught / reinforced	Assertions; finite-value checks; completion invariant.

representation	bandwidth_multiplier	comoving	raw	comoving_minus_raw
0	0.5	0.658550	0.484923	0.173627
1	1.0	0.189430	0.107103	0.082327
2	2.0	0.036751	0.019244	0.017507

Figure 14. Cell 30 bandwidth-sensitivity table.

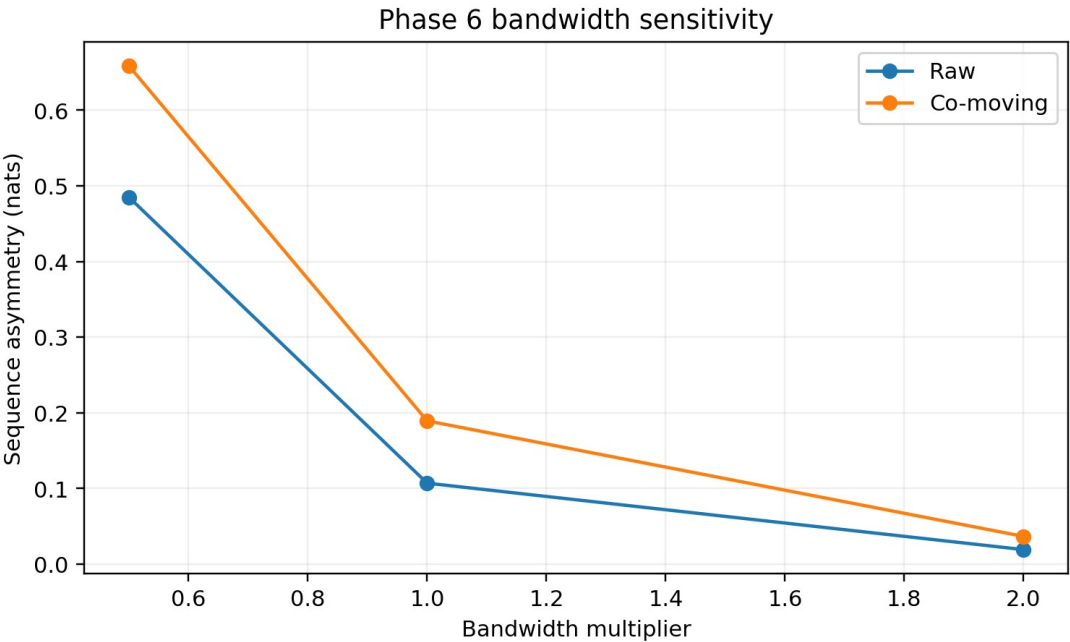


Figure 15. Bandwidth sensitivity: ordering persisted, absolute magnitude changed strongly.

	metric	bootstrap_mean	ci025	ci975
0	raw	0.223156	0.104131	0.478895
1	comoving	0.298824	0.166411	0.559712
2	comoving_minus_raw	0.075668	-0.033210	0.184291

Figure 16. Cell 32 cell-level bootstrap summary.

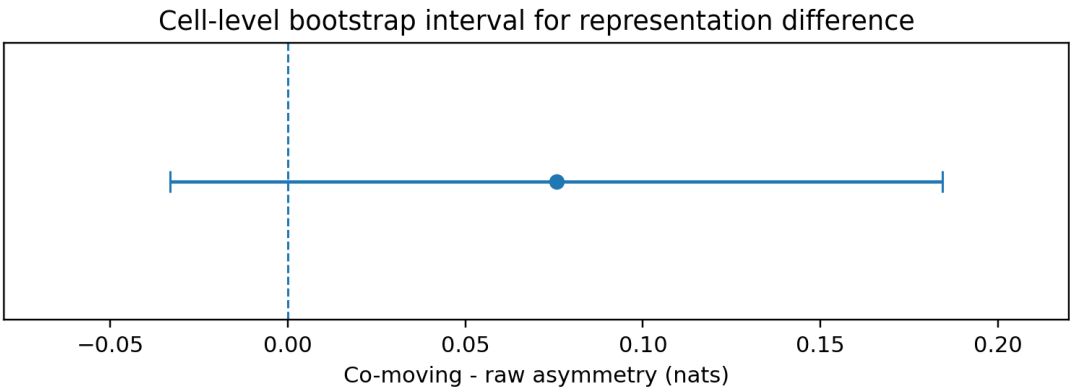


Figure 17. Bootstrap interval for co-moving minus raw crosses zero.

```
display(
  sequence_null_comparison
)
```

	representation	observed_asymmetry	null_mean	null_q95	observed_minus_null_mean	above_null_q95
0	raw	0.107103	0.190833	0.264116	-0.083730	False
1	comoving	0.189430	0.185367	0.261925	0.004063	False

Figure 18. Cell 34 observed-versus-null comparison table.

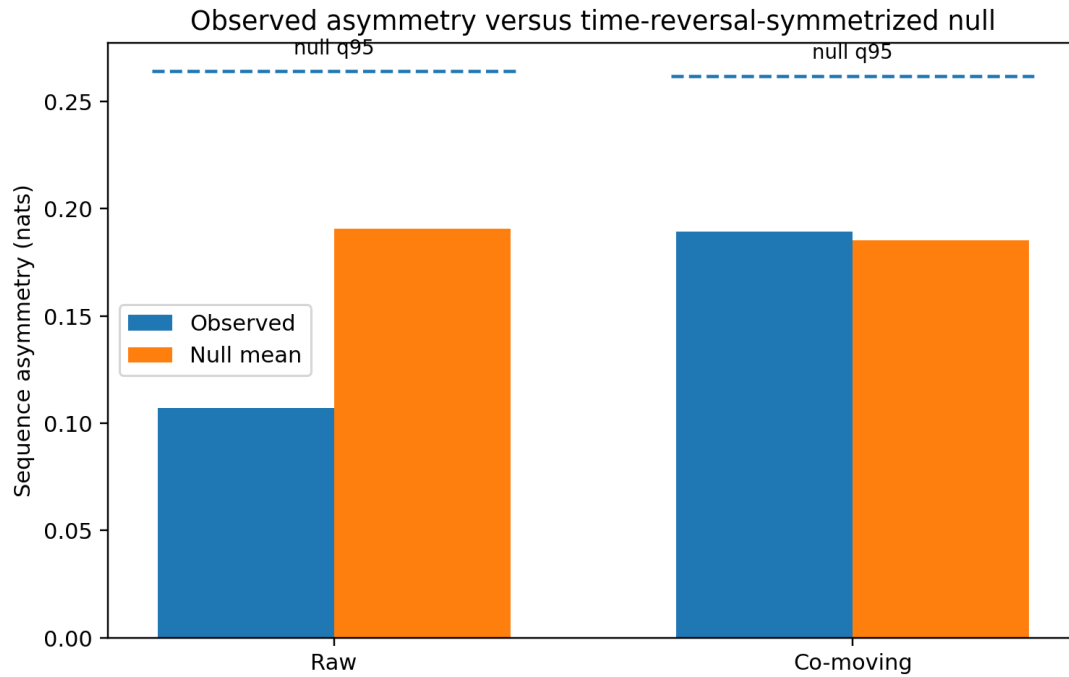


Figure 19. Neither observed asymmetry exceeds the null 95th percentile.

Phase 6 computational validation: PASSED

Figure 20. Cell 35 final computational validation checkpoint.

DECISIVE SCIENTIFIC RESULT

The null control overruled the temptation to interpret positive KL values as evidence of a time arrow. Raw observed asymmetry was below the null mean; co-moving observed asymmetry was essentially at the null mean; neither exceeded q95.

[↑ Back to Contents](#)

9. Cell 36: final Phase 6 scientific conclusion

Cell 36 — Final Phase 6 conclusion

Type / role	Markdown / scientific synthesis
Code or content summary	Documented sequence construction, time-reversal validation, feature representation, baseline asymmetry, bootstrap, null result, and interpretation limits.
Stored output	No executable output.
Interpretation / decision	Phase 6 ended with a scientifically valid non-detection: the selected representation did not show asymmetry beyond finite-sample/KDE null background. This is not evidence that the true dynamics are reversible.
Concepts taught / reinforced	Scientific writing; null-result interpretation; distinction between non-detection and proof of absence.

Final notebook wording - core meaning

Phase 6 successfully established a reproducible framework for exact ordered sequence construction and correct time reversal. The selected low-dimensional feature representation did not show detectable time-reversal asymmetry beyond the finite-sample/KDE background. The result is a scientifically meaningful non-detection, not a failed project and not evidence that the true cellular dynamics are reversible.

INTERPRETATION LANGUAGE TO PRESERVE

Preferred wording: “Within the selected two-step feature representation and the current MSC01 dataset, the analysis did not detect sequence-level time-reversal asymmetry clearly exceeding the time-reversal-symmetrized null background.” Avoid: “The cells are reversible,” “there is no irreversibility,” or “entropy production is zero.”

[↑ Back to Contents](#)

10. Integrated Phase 6 results and interpretation

10.1 Core numerical summary

Quantity	Raw	Co-moving	Interpretation
Exact two-step sequences	392	392	Same exact temporal structure; 32 cells.
Valid turning-angle sequences	376	380	Zero-length steps make angle undefined; representations differ.
Paired valid analysis set	365	365	Same sequences in both; 31 cells.
Baseline asymmetry (nats)	0.107103	0.189430	Positive estimator values only; not yet evidence.
Null mean	0.190833	0.185367	Finite-sample/KDE background is itself positive.
Null q95	0.264116	0.261925	Observed values are below q95.
Observed above null q95?	No	No	No detectable signal beyond chosen null.

10.2 What bandwidth sensitivity added

At half the baseline bandwidth, asymmetry values rose to roughly 0.485 (raw) and 0.659 (co-moving); at double bandwidth they fell to roughly 0.019 and 0.037. This large numerical range shows that the absolute KL estimate is a smoothing-dependent statistic in this finite dataset. The ordering co-moving > raw survived the tested range, but that alone was not enough to support a biological claim.

10.3 What the cell bootstrap added

The 95% exploratory bootstrap interval for co-moving minus raw was [-0.033, 0.184] nats. Because zero lies inside this interval, the representation difference is sensitive to which cells are represented. This prevents overinterpreting the baseline ordering as a stable population-level effect.

10.4 What the null control added

The null control is the most important result of the phase. It demonstrates that a KDE-based KL asymmetry around 0.18–0.19 nats can arise even after time directionality is removed in expectation. Therefore, “positive KL” and “detectable time-reversal asymmetry” are not the same statement.

10.5 Relation to Phase 5

Phase 5	Phase 6
Single displacement Δr	Ordered pair $(\Delta r_1, \Delta r_2)$
Compare $p(\Delta r)$ with $p(-\Delta r)$	Compare $p(z)$ with $p(-z)$, where z is validated time-odd sequence features
Spatial inversion can reflect common directionality	Explicit reversal uses temporal order
Some observed estimates exceeded simple inversion-symmetric null q95	Neither sequence-feature estimate exceeded time-reversal-symmetrized null q95
Exploratory evidence for single-step spatial inversion asymmetry	No detectable sequence-level asymmetry in the selected 2D feature projection

The project became more conservative, not less successful. Phase 6 showed why a strong workflow needs a null reference: the method was capable of producing a positive number even when the null was deliberately symmetrized. The correct conclusion is therefore a calibrated non-detection.

[↑ Back to Contents](#)

11. Decision logic and project mindset

11.1 Phase 6 operating principle

The project is a scientifically serious training and portfolio project. It should demonstrate correct thinking, transparent limitations, reproducible code, and honest inference. It does not need to produce a paper-ready biological discovery or a statistically significant positive result.

11.2 When to stop and fix

- A structural error that changes the scientific object - for example, a wrong lag or non-consecutive sequence - must stop the workflow and be fixed.
- A mathematical inconsistency - for example, a reversal operator that fails $R(R(S))=S$ - must stop the workflow.
- A comparison using different raw and co-moving sequence identities would invalidate paired interpretation and must be corrected.

11.3 When to document and continue

- Zero-length steps causing undefined turning angles are a mathematical limitation; mark them invalid and use a paired valid subset.
- Different valid-turn counts after common-motion subtraction are not automatically tracking failures; the transformation changes step vectors.
- A positive KL estimate does not trigger a search for a biological story; run the null control first.
- A null non-detection does not trigger extra analyses merely to seek significance. Record the limitation and move to the next planned question.

11.4 Decisions explicitly rejected

Rejected direction	Why it was rejected in Phase 6
Direct 4D KDE as the main analysis	392 sequences are limited for a data-hungry four-dimensional density estimate; lower-dimensional features were more interpretable for this training phase.
Using raw turning angle in Euclidean KDE	Circular boundary at $\pm\pi$ is inappropriate for ordinary Euclidean distance.
Mean-centering features before reversal comparison	Would shift the physically meaningful reversal origin and complicate $z \rightarrow -z$.
Treating every sequence row as independent in bootstrap	Overlapping sequences within a cell are dependent; cell-level resampling is more defensible.
Interpreting co-moving > raw as a real biological effect	Bootstrap difference crossed zero and null comparison did not support a signal.
Adding more diagnostics after null non-detection	Would make the phase heavier without changing the answer to the planned question.

[↑ Back to Contents](#)

12. Python and scientific-computing concepts learned

Concept	Where it appeared	Why it matters
Reusable functions	sequence constructor, reversal, feature extraction, scaling, KDE, metric	Turns notebook logic into testable scientific operations.
DataFrame merge with exact keys	Cell 5	Encodes temporal continuity explicitly instead of relying on row order.
<code>.rename()</code> + column selection + <code>.copy()</code>	Cell 5	Creates semantic first-step and second-step tables safely.
NumPy slicing <code>[:, i]</code>	Cells 10 and 13	Efficient component-wise vectorized operations.
<code>np.column_stack</code>	Cells 10 and 23	Builds structured numeric feature arrays.
<code>np.hypot</code>	Cell 13	Stable vector magnitude calculation.
<code>np.arctan2(cross, dot)</code>	Cell 13	Signed angle with correct quadrant information.
Boolean masks + <code>np.where</code>	Cell 13	Represent mathematically invalid angles as missing values.
<code>np.sin</code> and circular-variable reasoning	Cells 15–19	Transforms signed turning information into a KDE-friendly time-odd scalar.
Robust scaling	Cell 22	Balances heterogeneous feature units without centering the reversal origin.
KDE in scikit-learn	Cells 25–28	Estimates smooth 2D feature densities.
Symmetric grid indexing	Cell 26	Makes p(-z) correspond to <code>[:, -1, ::-1]</code> exactly.
Cluster bootstrap	Cell 31	Resamples cells rather than treating correlated within-cell sequences as IID.
<code>groupby().agg()</code> + lambda quantile	Cell 34	Summarizes null distributions compactly.
Assertions as scientific invariants	Cells 7, 11, 14, 19, 24, 35	Makes mathematical and structural assumptions executable.

A useful programming lesson from Phase 6

A notebook can remain educational without becoming enormous. Phase 6 reused patterns learned in Phase 5, explained the genuinely new concepts in detail, and stopped at 36 cells. This is a better long-term project architecture than re-teaching every familiar operation from scratch.

[↑ Back to Contents](#)

13. Reproducibility, testing, and Git workflow

13.1 Environment and tests

```
conda activate cell-irreversibility
python -m pytest -q
```

RECORDED VALIDATION OUTPUT

```
..... [100%]
9 passed in 0.55s
```

The automated suite primarily protects the earlier exact-lag trajectory machinery. Phase 6 adds notebook-level assertion checkpoints for its sequence-specific mathematics and data invariants rather than expanding the unit-test suite solely for exploratory notebook helpers.

13.2 Git workflow recorded during Phase 6

Step	Recorded state
Working branch	`phase-6-sequence-irreversibility`
Phase 5 handbook commit on the branch	`83f32f0` — Add standalone Phase 5 handbook
Phase 6 notebook commit	`f26276a` — Complete Phase 6 sequence-level irreversibility analysis
Push	Branch pushed with upstream tracking to `origin/phase-6-sequence-irreversibility`
Clean status after push	Local branch synchronized with origin; no untracked files.
PR/merge workflow	Performed next in the established project workflow before README update work.

13.3 Repository file introduced

```
notebooks/06_phase6_sequence_irreversibility_MSC01.ipynb
```

The naming convention intentionally includes `phase6` so that future multi-notebook phases can remain visibly grouped by phase. The default preference remains one notebook per phase when manageable.

[↑ Back to Contents](#)

14. Phase 7 handoff and next decision space

Phase 7 should not begin by assuming Phase 6 “failed.” Phase 6 answered its planned question: the selected two-step, two-feature representation did not exceed its time-reversal-symmetrized null background. The next phase should ask whether richer temporal information can be tested without sacrificing interpretability or overwhelming the available sample size.

Candidate directions

- Use richer representations of the original 4D two-step sequence without direct high-dimensional KDE, for example classification-based forward-vs-reversed discrimination with strict cell-level cross-validation.
- Use longer ordered sequences if sample count remains adequate, while explicitly enforcing exact continuity.
- Construct alternative physically interpretable time-odd observables beyond Δm and signed turning.
- Use methods that directly estimate distinguishability rather than requiring a dense high-dimensional probability estimate.

Phase 7 design constraints that must remain locked

- Raw and co-moving representations remain parallel sensitivity views; co-moving is not automatically “truth.”
- Sequence identities and temporal continuity must be exact and paired.
- Any method must include an appropriate null or out-of-sample control before positive scores are interpreted.
- Cell-level dependence must be respected in resampling or cross-validation.
- The objective is educational scientific rigor and a defensible GitHub portfolio, not a forced positive result.

RECOMMENDED NEXT STARTING QUESTION

Can a richer representation of exact ordered sequences distinguish forward from time-reversed examples out of sample, at the cell level, better than a label-symmetrized or time-reversal null?

[↑ Back to Contents](#)

Appendix A. Key Phase 6 code listings

These listings preserve the core algorithmic logic needed to reconstruct the phase. The executed notebook remains the authoritative runnable artifact.

A1. Exact two-step sequence constructor

```
def build_two_step_sequences(tracks):
    validate_tracks(tracks)
    one_frame_steps = compute_steps_for_tau(tracks, tau_frames=1)

    first_steps = one_frame_steps.rename(columns={
        "frame_end": "frame_mid", "t_end_min": "t_mid_min",
        "dx_um": "dx1_um", "dy_um": "dy1_um",
    })[["cell_id", "frame_start", "t_start_min", "frame_mid", "t_mid_min", "dx1_um", "dy1_um"]].copy()

    second_steps = one_frame_steps.rename(columns={
        "frame_start": "frame_mid", "dx_um": "dx2_um", "dy_um": "dy2_um",
    })[["cell_id", "frame_mid", "frame_end", "t_end_min", "dx2_um", "dy2_um"]].copy()

    sequences = first_steps.merge(second_steps, on=["cell_id", "frame_mid"],
                                   how="inner", validate="one_to_one")
    # explicit checks require start -> mid -> end to advance by exactly one frame each
    return sequences.sort_values(["cell_id", "frame_start"]).reset_index(drop=True)
```

A2. Time-reversal operator

```
def reverse_two_step_sequences(sequence_array):
    sequence_array = np.asarray(sequence_array, dtype=float)
    if sequence_array.ndim != 2 or sequence_array.shape[1] != 4:
        raise ValueError("Expected an array with shape (n, 4).")
    return np.column_stack([
        -sequence_array[:, 2], -sequence_array[:, 3],
        -sequence_array[:, 0], -sequence_array[:, 1],
    ])
```

A3. Two-step feature extractor

```
dx1, dy1, dx2, dy2 = [sequence_array[:, i] for i in range(4)]
magnitude1 = np.hypot(dx1, dy1)
magnitude2 = np.hypot(dx2, dy2)
delta_magnitude = magnitude2 - magnitude1
cross_term = dx1 * dy2 - dy1 * dx2
dot_term = dx1 * dx2 + dy1 * dy2
turning_angle_rad = np.arctan2(cross_term, dot_term)
valid_turn = (magnitude1 > 0) & (magnitude2 > 0)
turning_angle_rad = np.where(valid_turn, turning_angle_rad, np.nan)
```

A4. Robust pooled scaling without centering

```
def robust_scale_1d(values):
    values = np.asarray(values, dtype=float)
    values = values[np.isfinite(values)]
    std = np.std(values, ddof=1)
    q25, q75 = np.quantile(values, [0.25, 0.75])
    iqr_scale = (q75 - q25) / 1.34
    candidates = [s for s in [std, iqr_scale] if np.isfinite(s) and s > 0]
    if not candidates:
        raise ValueError("Could not determine a positive feature scale.")
    return float(min(candidates))

# z_scaled = [delta_magnitude / pooled_scale_dm, sin(theta) / pooled_scale_sin]
# No mean centering: the reversal origin remains at zero.
```

A5. Sequence asymmetry index

```
def sequence_asymmetry_index(density, x_grid, y_grid):
    cell_area = (x_grid[1]-x_grid[0]) * (y_grid[1]-y_grid[0])
    grid_mass = density.sum() * cell_area
    p = density / grid_mass
    p_reversed = p[::-1, ::-1]
    epsilon = p.max() * 1e-12
    p_safe = p + epsilon
    q_safe = p_reversed + epsilon
    p_safe /= p_safe.sum() * cell_area
    q_safe /= q_safe.sum() * cell_area
    asymmetry = (p_safe * np.log(p_safe / q_safe)).sum() * cell_area
    return float(asymmetry), float(grid_mass)
```

A6. Cell-level bootstrap principle

```
sampled_cells = rng.choice(BOOTSTRAP_CELLS, size=len(BOOTSTRAP_CELLS), replace=True)
raw_blocks, comoving_blocks = [], []
for cell_id in sampled_cells:
    raw_blocks.append(raw_feature_array[raw_analysis_features["cell_id"].eq(cell_id).to_numpy()])
    comoving_blocks.append(comoving_feature_array[comoving_analysis_features["cell_id"].eq(cell_id).to_numpy()])
```

```
raw_bootstrap = np.vstack(raw_blocks)
comoving_bootstrap = np.vstack(comoving_blocks)
```

A7. Time-reversal-symmetrized null principle

```
signs = null_rng.choice([-1.0, 1.0], size=(len(raw_feature_array), 1))
null_raw = raw_feature_array * signs
null_comoving = comoving_feature_array * signs
# Same sign realization is used for paired raw and co-moving sequences.
```

[↑ Back to Contents](#)

Appendix B. Exact numerical result ledger

Result	Value
Input rows per representation	474
Exact two-step sequences per representation	392
Cells contributing exact sequences	32
Raw valid turning angles	376 / 392
Co-moving valid turning angles	380 / 392
Paired valid analysis sequences	365 / 392
Cells in paired analysis	31
Pooled Δm scale	1.649157496997992
Pooled sin(turn) scale	0.49157537528456907
Baseline bandwidth	0.374069
Raw grid mass	0.999935
Co-moving grid mass	0.999890
Raw baseline sequence asymmetry	0.107103 nats
Co-moving baseline sequence asymmetry	0.189430 nats
Bootstrap replicates	100
Bootstrap difference mean	0.075668 nats
Bootstrap difference 95% interval	[-0.033210, 0.184291] nats
Null simulations	100 per representation (200 stored results)
Raw null mean / q95	0.190833 / 0.264116
Co-moving null mean / q95	0.185367 / 0.261925
Raw above null q95	False
Co-moving above null q95	False
Automated test status	9 passed in 0.55 s

Bandwidth sensitivity ledger

Multiplier	Raw	Co-moving	Co-moving - raw
0.5	0.484923	0.658550	0.173627
1.0	0.107103	0.189430	0.082327
2.0	0.019244	0.036751	0.017507

Bootstrap summary ledger

Metric	Mean	2.5%	97.5%
Raw	0.223156	0.104131	0.478895
Co-moving	0.298824	0.166411	0.559712
Co-moving - raw	0.075668	-0.033210	0.184291

Null comparison ledger

Representation	Observed	Null mean	Null q95	Observed-null mean	Above q95?
Raw	0.107103	0.190833	0.264116	-0.083730	False
Co-moving	0.189430	0.185367	0.261925	0.004063	False

[↑ Back to Contents](#)

Appendix C. Glossary and notation

Term / symbol	Meaning
$r(t)$	2D cell position at frame/time t .
$\Delta r_1, \Delta r_2$	First and second consecutive one-frame displacement vectors in a two-step sequence.
S	Ordered forward sequence $(\Delta r_1, \Delta r_2)$.
$R(S)$	Correct time-reversed sequence $(-\Delta r_2, -\Delta r_1)$.
Involution	An operation that returns the original object when applied twice: $R(R(S))=S$.
m_1, m_2	Magnitudes of the first and second displacement vectors.
Δm	$m_2 - m_1$; a time-odd step-size-change feature.
θ	Signed turning angle from Δr_1 to Δr_2 .
$\sin\theta$	KDE-friendly signed-turn representation used in Phase 6.
z	2D feature vector $(\Delta m, \sin\theta)$.
Time-odd	A quantity that changes sign under time reversal.
KDE	Kernel density estimation; a smooth estimate of a probability density.
Bandwidth h	KDE smoothing scale; Phase 6 used a 2D rule-of-thumb baseline and sensitivity factors 0.5, 1, 2.
KL divergence	Relative-entropy-like divergence $D_{KL}[p q]$; used here as a distributional asymmetry score.
Grid mass	Numerical integral of the KDE over the evaluation grid; should be close to 1.
Cell-level bootstrap	Resampling cells with replacement while retaining all selected sequences from each sampled cell.
Time-reversal-symmetrized null	Randomly replace each z by z or $-z$ with equal probability, removing systematic directionality in expectation.
Non-detection	Observed metric does not exceed the chosen null reference; not proof that the underlying process is reversible.

[↑ Back to Contents](#)

Appendix D. New-chat handoff prompt

Use the following prompt in a fresh ChatGPT conversation together with the Phase 0–4 Master Handbook, Phase 5 Standalone Handbook, and this Phase 6 Handbook. The goal is continuity without requiring the new chat to reconstruct old decisions from memory.

We are continuing the Cell Motility Irreversibility project. Treat the attached Phase 0–4 Master Handbook, Phase 5 Standalone Handbook, and Phase 6 Standalone Handbook as the authoritative project record.

Project purpose: an educational, reproducible computational biophysics project and GitHub portfolio. It should be scientifically rigorous and transparent, but it is not required to produce a publication, a positive irreversibility result, or a thermodynamic entropy-production claim.

Locked status through Phase 6:

- Dataset: Fluo-C2DL-MSD sequence 01; 0.3 $\mu\text{m}/\text{pixel}$; 20 min/frame; 48 frames; 35 cleaned tracks.
- Exact-lag implementation is authoritative; row-shift lag logic is forbidden.
- Raw and co-moving representations are parallel sensitivity views; co-moving is not assumed to be the true drift-free trajectory.
- Phase 5 found exploratory single-displacement spatial inversion asymmetry in parts of the analysis, but that was not full temporal irreversibility.
- Phase 6 constructed 392 exact two-step sequences from 32 cells and validated the correct time-reversal operator $R(S)=(-\Delta r_2, -\Delta r_1)$, including $R(R(S))=S$.
- Primary Phase 6 feature representation: $z=(\Delta m, \sin(\theta))$; both components were validated to reverse sign.
- Paired valid analysis subset: 365 sequences from 31 cells.
- Baseline sequence asymmetry: raw 0.107103 nats, co-moving 0.189430 nats.
- Cell-level bootstrap difference (co-moving - raw): mean 0.075668, interval $[-0.033210, 0.184291]$, so the representation difference is not robust.
- Time-reversal-symmetrized null: raw null mean/q95 0.190833/0.264116; co-moving 0.185367/0.261925; neither observed estimate exceeded q95.
- Final Phase 6 conclusion: within this two-step, two-feature representation, no sequence-level time-reversal asymmetry was detected beyond finite-sample/KDE null background. This is not evidence that the true cell dynamics are reversible.
- Tests remained 9 passed. Notebook: notebooks/06_phase6_sequence_irreversibility_MSC01.ipynb.

Operating mindset:

- Fix errors that invalidate the scientific object or mathematics.
- For limited ambiguities, document them and use a simple sensitivity analysis if needed; do not open forensic detours automatically.
- Respect cell-level dependence in bootstrap/cross-validation.
- Never interpret a positive estimator before comparing it with an appropriate null or out-of-sample control.
- Do not force a positive result.

For Phase 7, first decide on the smallest scientifically meaningful extension beyond the current reduced two-step representation. Prefer one coherent notebook for the phase if manageable, with filenames that visibly include the phase number. Keep the educational explanations of genuinely new Python, mathematics, statistics, and biophysics concepts, but reuse familiar patterns instead of expanding the notebook unnecessarily.

[↑ Back to Contents](#)